

Memoria del TFG: PLANTAFACIL

Portada

•

Título del proyecto: PLANTAFACIL

•

Aplicación: TusPlantitas / PlantCare

•

Autor: [Nombre del estudiante]

•

Título del grado: [Grado en Ingeniería Informática / Electrónica / Telecomunicaciones]

•

Tutor: [Nombre del tutor]

•

Centro: [Nombre de la universidad]

•

Curso académico: 2024-2025

Resumen

Este trabajo de fin de grado presenta el desarrollo de PLANTAFACIL, una aplicación móvil multiplataforma construida en Flutter para facilitar el cuidado de plantas domésticas y de jardín.

La aplicación combina: gestión de plantas personales, catálogo de especies, recordatorios de riego, identificación automática de plantas mediante inteligencia artificial y sincronización con servicios en la nube de Firebase.

Introducción

Las aplicaciones de cuidado de plantas han ganado relevancia por la creciente demanda de asistentes digitales para el seguimiento de tareas de jardinería y cultivo. PLANTAFACIL aborda este problema ofreciendo una herramienta intuitiva, accesible y segura que ayuda a los usuarios a mantener hábitos

de riego adecuados y a conocer las características de cada planta.

Objetivos

1.

Diseñar e implementar una aplicación móvil multiplataforma en Flutter.

2.

Proveer funcionalidades de gestión de plantas y catálogo botánico.

3.

Añadir recordatorios inteligentes de riego con notificaciones locales.

4.

Integrar identificación de especies vegetales usando PlantNet a través de Cloud Functions.

5.

Usar persistencia local con SQLite y sincronización con Firebase.

6.

Garantizar una experiencia de usuario fluida con navegación moderna y temas claros/oscuro.

Herramientas y tecnologías

-

Flutter 3.x y Dart 3.x.

-

Firebase: Auth, Firestore, Storage, Functions.

-

SQLite con `sqflite` para datos locales.

-

Provider para gestión de estado.

-

`go_router` para navegación declarativa.

-

flutter_local_notifications para avisos de riego.

-

image_picker para capturar o seleccionar imágenes.

-

HTTP y Cloud Functions para la integración con PlantNet.

-

Node.js y Firebase Functions para la lógica del backend de identificación.

Arquitectura del sistema

El proyecto sigue una arquitectura en capas:

-

Capa de presentación: pantallas, widgets y providers.

-

Capa de lógica de negocio: repositorios, datasources y persistencia.

-

Capa de dominio: entidades y modelos de dominio.

-

Capa de servicios transversales: utilidades, rutas y servicios transversales.

-

Backend de Cloud Functions.

Esto facilita la separación de responsabilidades, la escalabilidad y la mantenibilidad.

Diseño e implementación

1. Entrada y configuración

El `AppDelegate` de `AppDelegate`, `AppDelegate` y el servicio de notificaciones antes de arrancar las dependencias principales se registran con `AppDelegate` a que los proveedores de estado que son disponibles en toda la aplicación.

2. Gestión de estado

Se utilizan los siguientes providers:

- `AuthProvider` controla autenticación con Firebase Auth.
- `AsyncStorage` gestiona las plantas del usuario y su persistencia SQLite.
- `Firestore` carga y filtra el catálogo de especies.
- `Firestore` lógica de cuestionarios botánicos.
- `AsyncStorage` gestiona temas y preferencias de la app.

3. Navegación

`Router` define rutas protegidas y públicas usando `firebaseAuth`. El enrutador redirige a la pantalla de inicio de sesión cuando el usuario no está autenticado y habilita rutas de catálogo, perfil, ajustes y gestión de plantas.

4. Persistencia local

`SQLite` crea y mantiene una base de datos SQLite con dos tablas principales:

- `plants`: datos de las plantas del usuario.
- `species`: catálogo predefinido de especies.

Se incluye un proceso de migración y actualización de la base de datos (versiones 2 y 3) para extender el esquema con `wateringAmount` y `plantName`.

Además, `loadInitialPlants()` lee datos de catálogo iniciales que se cargan se crea la base de datos.

5. Identificación de plantas

La identificación de plantas se realiza mediante $\text{AE } \text{çD-FVçF-f-6 F-öå6W}$ vice

-

Convierte imágenes seleccionadas a Base64.

-

Envía imágenes y órganos seleccionados a una Cloud Function llamada `-FVçF-g•Æ ç@`.

La función en `gVæ7F-öç2÷7&2÷ Æ çBÖ-FVçF-f-6 F-öâæ§0` actúa como proxy a la API de PlantNet, evitando problemas de CORS y protegiendo la clave de API.

6. Cloud Functions y seguridad

–FVçF–g• Æ ç@ valida:

-

Que el usuario esté autenticado.

-

que se envíen entre 1 y 5 imágenes.

-

que haya una lista de órganos del mismo tamaño que las imágenes.

Posteriormente construye un `ormData Â Vávia la solicitud a PlantNet y devuelve los resultados al cliente

7. Notificaciones y recordatorios

recordatorios de riego. Los usuarios reciben avisos en función de la frecuencia de riego configurada y del historial de cuidados.

8. Interfaz y experiencia de usuario

La app integra temas claros y oscuros mediante F+VÖP y permite elegir el modo desde 6WGF-æw5 &övider à

La UI dispone de pantallas para:

-

Inicio y resumen de plantas.

-

Catálogo de especies con búsquedas y filtros.

-

Gestión de plantas personales (añadir, editar, eliminar).

-

Identificación de plantas por imagen.

-

Cuestionarios botánicos de refuerzo.

-

Ajustes de usuario.

Funcionalidades clave

-

Registro e inicio de sesión con Firebase.

-

Gestión de plantas con atributos como nombre, especie, imagen, luz, riego, temperatura y humedad.

-

Catálogo botánico local con descripción, cuidados y fotografías.

-

Identificación automática de especies mediante análisis de imágenes.

-

Recordatorios de riego con notificaciones locales.

-

Alternancia de tema claro/oscuro.

-

Cuestionarios tipo quiz para aprendizaje sobre plantas.

Infraestructura backend

El backend usa Firebase Functions con Node.js 20 y las dependencias:

-

`firebase-functions`

-

`firebase-admin`

-

`firebase`

-

`firebase-data`

La función `firebase-functions` protege la clave de API y permite que la aplicación móvil solicite identificados vegetales sin exponer credenciales.

Seguridad y protección de datos

-

Firebase Auth protege el acceso a las funciones y a los datos del usuario.

-

La Cloud Function valida la autenticidad de la llamada.

-

SQLite almacena los datos del usuario localmente en el dispositivo para proteger la privacidad.

-

La aplicación no expone la clave de PlantNet en el cliente.

Pruebas y validación

Se han creado pruebas unitarias básicas dentro de FW7Bð para validar:

- lógica de proveedores y estados.
- funcionalidad de cuestionarios.
- comportamientos de valores de riego.

También se utiliza la lógica de Flutter para asegurar rutas y redirecciones correctas en función del estado de autenticación.

Resultados y logros

PLANTAFACIL demuestra que es posible construir una solución móvil completa basada en Flutter y Firebase que abarca:

- experiencia de usuario moderna.
- persistencia local y nube híbrida.
- integración con APIs externas de IA.
- notificaciones y recordatorios prácticos.

Conclusiones

El proyecto cumple los objetivos planteados y ofrece una base sólida para futuras mejoras, tales como

- sincronización en tiempo real entre dispositivos.

- mejores análisis de hábitos de cuidado.
- soporte multilinguaje.
- evidencia fotográfica del estado de las plantas.

Trabajo futuro

- Añadir historial de riego y estado de crecimiento.
- Implementar sincronización completa con Firestore.
- Incluir más proyectos/biomas para identificación de PlantNet.
- Crear un panel de estadísticas y recomendaciones personalizadas.

Anexos

- Archivo principal de Flutter: `Æ-"öÖ -âæF t à`
- Backend de identificación: `gVæ7F-öç2÷7&2÷ Æ çBÖ-FVçF-f-6 F-öâæ$0.`
- Persistencia SQLite: `Æ-"öF F öF F 6÷W&6W2öF F & 6Uö†VÇ W .dart à`
- Configuración de rutas: `Æ-"ö6÷&R÷WF-Ç2ö ÷&÷WFW .dart à`

-

Servicio de identificación: Æ—"ö6÷&R÷WF—Ç2÷ Æ çEö—FVçF—f—6 F—öå÷6W vice.dart à

-

Dependencias principales: V'7 V2ç•aml à